a) Max and Min

1. Pseudocode: <provided in github code>
2. Recurrence: C(n) = 2C(n/2) + 2 for n > 2, with C(2) = 1 and C(1) = 0. Each call makes two recursive calls, then one comparison to combine the maxima and one to combine the minima. Expanding with n = 2^k gives C(n) = 2^(k-1) + 2^k - 2 = 3n/2 - 2 comparisons.
3. Brute force makes 2(n-1) = 2n - 2 comparisons, so divide and conquer is about 25% better in comparison count. Both are Θ(n). The recursion also adds call overhead and uses O(log n) stack space.

b) Exponentiation a^n

1. Pseudocode: <provided in github code>
2. Recurrence: M(n) = M(n/2) + 1 for n > 1 when n is even, using a^n = (a^(n/2))^2, with M(1) = 0. For odd n, a^n = a * (a^((n-1)/2))^2, which costs up to 2 multiplications per level. Solving gives M(n) = log2(n) for n = 2^k, and between log2(n) and 2·log2(n) in general. This is Θ(log n).
3. Brute force multiplies a by itself n-1 times, which is Θ(n). Divide and conquer is exponentially faster. For n = 1,000,000 it takes about 20 to 40 multiplications versus 999,999.

c) Counting Inversions

1. Algorithm: <provided in github code>

Idea: it is a modified merge sort. Split the array in half and recursively count the inversions in each half while sorting them. During the merge, whenever an element from the right half is placed before the remaining elements of the left half, add the number of those remaining left elements to the count. Total inversions = left + right + cross counts. The recurrence is T(n) = 2T(n/2) + O(n), so the time is O(n log n).

d) Quicksort Complexity

1. Worst case is $\Theta(n^2)$. It happens when the pivot is always the smallest or largest element, such as an already sorted array with first-element pivoting, giving splits of sizes n-1 and 0. Best case is Θ(n log n). It happens when every pivot splits the array into two equal halves, giving T(n) = 2T(n/2) + n. The average case is also Θ(n log n), about 1.39 n log2 n comparisons.

e) 1D Closest Pair

1. Algorithm: <provided in github code>

Idea: find the median (or split the array into left and right halves by value). Recursively find the closest pair in the left half and in the right half. The only cross-pair candidate is the maximum of the left half and the minimum of the right half. Return the smallest of the three distances. Recurrence: T(n) = 2T(n/2) + O(n) for the split and combine, so the efficiency class is Θ(n log n).

2. It is not a good algorithm for this problem. Simply sort the array in Θ(n log n) and scan adjacent pairs in Θ(n), which is the same asymptotic cost with a much simpler implementation. The divide and conquer version has no advantage over it. Sorting is also a lower-bound-matching approach in the comparison model.

f) Solved Exercise 1: Median of two databases

Problem: two databases hold n distinct values each, and a query returns the k-th smallest value of one database. Find the median of the 2n values using O(log n) queries.

1. Algorithm: <provided in github code>

Idea: query the median (k = n/2) of each database, giving a and b. If a < b, the overall median cannot be in the lower half of A or the upper half of B, so discard both. If b < a, discard the opposite halves. Recurse on the remaining n/2 values in each. For n = 1, return the smaller value. Recurrence: T(n) = T(n/2) + 2 queries, so O(log n) queries.

g) Solved Exercise 2: Significant inversions

Problem: count pairs i < j with $a_i > 2 \cdot a_j$ in O(n log n).

1. Algorithm: <provided in github code>

Idea: modified merge sort. Recursively sort each half and count its significant inversions. Before merging, count cross pairs with two pointers: for each element b in the sorted right half, advance a pointer in the sorted left half while left > 2b, and add the number of left elements beyond the pointer. Then do the normal merge. Counting is O(n) per level, so T(n) = 2T(n/2) + O(n) = O(n log n).

h) Any three exercises (Exercises 1-7, pages 246-249)

Check the exact numbering against your edition. These are identified by problem.

Bank cards (majority equivalence class)

1. Algorithm: <provided in github code>

Problem: given n cards and a test that says whether two are equivalent, decide if more than n/2 are equivalent, using O(n log n) tests. Idea: split the cards in half and recursively find a majority card in each half. For each candidate (at most 2), compare it with all other cards and count matches. If a count exceeds n/2, that card represents the majority. Combining costs O(n) tests, so T(n) = 2T(n/2) + O(n) = O(n log n).

Local minimum in a complete binary tree

2. Algorithm: <provided in github code>

Problem: a complete binary tree has n = 2^d - 1 distinct labels. Find a node smaller than all its neighbors using O(log n) probes. Idea: probe the root and its two children. If the root is smaller than both, it is a local minimum. Otherwise move to the smaller child. That child is smaller than its parent, so a local minimum must exist in its subtree. Repeat down the tree. Each level costs at most 3 probes over d levels, so O(log n) probes.

Peak of an increasing-then-decreasing array

3. Algorithm: <provided in github code>

Problem: A increases to a peak and then decreases. Find the peak in O(log n). Idea: binary search. Compare A[mid] with A[mid+1]. If A[mid] < A[mid+1], the peak lies to the right, so move low to mid+1. Otherwise the peak is at mid or to the left, so move high to mid. Stop when low equals high. Each step halves the range, so T(n) = T(n/2) + O(1) = O(log n).